

AI-overdragelse — SF Middelfart nyhedsbrevsrobot

Repo: Mweimar2000/maja-sf.dk · Genereret 2026-09-07

Formål: Dette dokument gør det muligt for et andet menneske eller en anden AI-agent at forstå, gennemgå og genskabe arbejdet på denne robot uden adgang til den oprindelige chattråd. Det beskriver systemet, de fejl der er fundet og rettet, de invarianter der ikke må brydes, og hvordan man verificerer at alt virker.

Status ved overdragelse: Robotten kører, men den sidste rettelse (reservemodeller + token-plads, commit `2cfa55c`) er **ikke afprøvet i produktion**. Se afsnittet "Uafklarede punkter".

1. Hvad systemet er

En Google Apps Script-robot der automatisk skriver et ugentligt nyhedsbrev for partiet SF Middelfart. Den henter kommunale dagsordener og referater, lader en AI-model vurdere og analysere hver sag, skriver et nyhedsbrev i partiets faste tone, og **fakta-tjekker det mod kilden** før det lægges som kladde i Google Drive.

Element	Værdi
Repo	<code>github.com/Mweimar2000/maja-sf.dk</code>
Hovedfil	<code>sf-middelfart-robot-v8.gs</code> (~2570 linjer, ren JavaScript til Apps Script V8)
Tone-definition	<code>stilguide.md</code> (hentes LIVE fra GitHub ved hver kørsel)
Datakilde	FirstAgenda API på <code>https://dagsordener.middelfart.dk</code> (anonym auth)
AI-model	<code>gemini-3.7-flash</code> med fallback til <code>gemini-3.6-flash</code> , <code>gemini-3.5-flash</code>
Datalager	Google Sheet, kolonne A-O
Modtager	Én person (Maja); kladden lander i en Drive-mappe + notifikationsmail

Datastrøm

dagsordener.middelfart.dk (FirstAgenda API)

```
|
|─ ingestFromFirstAgendaApi()      ← daglig trigger kl. 12
|   auth → udvalgsliste → møder → dagsordenspunkter
|   skriver rækker i arket (A-I udfyldt, J-0 tom)
|   → analyzeNewRows_()  kalder Gemini pr. række, udfylder J-0
|
|─ ingestInboxEmails()             ← samme trigger, supplerende kilde (Gmail)
|
|─ dailyRepairAnalyses()           ← daglig trigger kl. 14
|   analyzePendingRows_() finder rækker uden analyse og retter dem
|
|─ generateWeeklyDraft()           ← ugentlig trigger lørdag kl. 13
|   læser ugens rækker → opdeler efter score
|   → generateNewsletterWithGemini_() skriver nyhedsbrevet
|   → createDraftDocument_()      GEMMER STRAKS (med advarselsbanner)
|   → collectGroundTruth_()       genhenter kildetekst fra API'et
|   → factCheckNewsletter_()      verificerer påstande mod kilden
|   → opdaterer dokumentet med fakta-tjek-rapporten øverst
|   → GmailApp.sendEmail()        notifikation med status i emnelinjen
```

Arkets kolonner

Kol	Felt	Skrives af
A	Modtaget (mødedato)	indsamling
B	Type (Dagsorden/Referat)	indsamling
C	Udvalg	indsamling
D	Emne	indsamling
E	Fra (kilde)	indsamling
F	ID (FA:<mødeId>:<punktId> eller Gmail message-id)	indsamling — bruges til dedup
G	URL	indsamling
H	Snippet (op til 8000 tegn kildetekst)	indsamling
I	Bilag	indsamling
J	TLDR	analyse
K	SF-analyse	analyse
L	Konkrete fakta	analyse
M	Beløb/tal	analyse
N	Score 1-5	analyse — styrer om sagen kommer i nyhedsbrevet
O	Programmatch	analyse

Rækkens fire mulige tilstande — vigtigt at kunne skelne

Tilstand	Kolonne J	Kolonne N	Betydning
Analyseret	rigtig tekst	1-5	færdig
Ægte formalia	Formalia/procedurepunkt	1	med vilje udeladt af nyhedsbrevet
Aldrig analyseret	tom	tom	venter på dailyRepairAnalyses()
Forgiftet (gammel fejl)	Analyse fejlede / Kunne ikke analyseres	1	skal repareres

Score styrer opdelingen: **>=4** top-sager, **=3** mellem-sager, **<=2** administrative (udelades af prosaen), **tom** = mangler analyse (udelades helt + advarsel).

2. Invarianter — bryd ikke disse

Disse er ikke stilpræferencer. Hver enkelt svarer til en fejl der faktisk er sket.

- 1. **Fakta-tjekket skal bevares.** Brugeren har eksplicit krævet at nyhedsbrevet er faktuel korrekt. Det må kun springes over hvis alternativet er at miste hele kørslen — og skal da markeres højlydt i både dokument og emnelinje.
- 2. `callGeminiJson_` og `callGeminiWithPdf_` skal returnere en **STRENG**. De sender videre til `parseJsonSafe_()` . Returnerer de et objekt, kaster `JSON.parse("[object Object]")` , og **hver eneste række bliver til en fejl**. Dette blev fanget i review, inden det ramte produktion.
- 3. **En fejl må aldrig blive til data.** Fejler en analyse, skal J-O stå TOM. Skriv aldrig en sentinel-score. `score: 1` gør sagen usynlig for altid, og `-1` virker heller ikke (`-1 <= 2` er sandt i JavaScript → lander i admin).
- 4. **Tom score betyder "aldrig analyseret", ikke 1.** `Number("") || 1` var præcis den fejl der skjulte 16 uanalyserede sager som "administrative".
- 5. **6-minutters grænsen er hård.** Alt retry skal gå gennem tidsbudgettet (`timeFor_`). Start aldrig et netværksskald der ikke kan nå at blive færdigt.
- 6. `generateNewsletterWithGemini_` returnerer **null ved fejl** — ikke en fejlstreng, og den kaster ikke. `generateWeeklyDraft` bruger `null` til at sende den danske fejlmail. Kaster man i stedet, får brugeren kun et engelsk stacktrace.
- 7. **Alt brugervendt er på dansk** — logbeskeder, mails, dokumentindhold. Kodekommentarer er også danske i denne fil.
- 8. **Skriv resultater pr. række, ikke samlet til sidst.** En batch-skrivning efter løkken taber 100 % af arbejdet ved timeout.

3. Kronologi — hvad der blev fundet og rettet

Ældst først. Commit-hash i parentes.

Fase 1 — tone og fakta-grundlag

#	Problem	Rettelse
1	Tonen i koden matchede ikke brugerens stilguide	<code>SF_TONE_GUIDE_FALLBACK</code> synkroniseret med <code>stilguide.md</code> ; stilguiden hentes nu LIVE fra GitHub med 1 times cache (<code>778a037</code> , <code>6616919</code>)
2	Intet tjek af om nyhedsbrevet var faktuel korrekt	Automatisk fakta-tjek indført: genhent kildetekst fra API'et → Gemini verificerer hver påstand → rapport øverst i dokumentet (<code>06ff966</code>)
3	<code>allAmounts</code> blev brugt i prompten men var aldrig defineret → <code>ReferenceError</code> ved hver kørsel	Variablen udtrækkes nu fra top- og mellemsager (<code>65663c9</code>)

Fase 2 — timeout og afskæring

#	Problem	Rodårsag	Rettelse
4	"Exceeded maximum execution time"	<code>testManualRun()</code> kørte indsamling + emails + nyhedsbrev i ét 6-min vindue, og <code>generateWeeklyDraft</code> lavede to separate FirstAgenda-auth	Splittet i <code>testManualRun()</code> + <code>testGenerateNewsletter()</code> ; auth genbruges via cookies (<code>c65c715</code>)
5	Nyhedsbrevet stoppede midt i en sætning	<code>maxOutputTokens: 4000</code> var for lavt, og <code>finishReason</code> blev aldrig tjekket	Hævet til 16384 + <code>finishReason</code> -tjek + advarsel hvis SEKTION 7 mangler (<code>1905ed6</code>)

Fase 3 — hallucination

#	Problem	Rettelse
6	Gemini opdigtede fakta ("97 % af kommunerne bruger FirstAgenda", "etableret 2011")	Prompten krævede "3-5 punkter" og "4-7 nøgletal" — så modellen fyldte op med sin egen viden. Reglerne skærpet, og sektionerne gjort fleksible: spring dem HELT over ved manglende data. Et ærligt kort nyhedsbrev er bedre end et langt med opdigtede tal (<code>c19b595</code>)
7	Fakta-tjekket flagede korrekte kalendermøder som "uverificerede"	<code>collectGroundTruth_</code> samlede kun facit for sager, ikke for kalenderen. Kommende møder tilføjes nu som kilde af typen <code>kalender</code> (<code>33b413d</code>)

Fase 4 — systematisk kodegennemgang (`85db84d`)

Fem fejl og fire finpudsninger. De to vigtigste:

- **Referater blev aldrig brugt.** Når et møde var afholdt og referatet udkom,

blev det filtreret væk som duplikat (samme ID). Nyhedsbrevet byggede altså altid på dagsordensteksten *fra før mødet* — aldrig de faktiske beslutninger. Nu opdateres rækken og analyseres på ny.

- **Fejlttekst kunne udsendes som nyhedsbrev.** Fejlede Gemini, blev strengen

"Fejl ved generering..." fakta-tjekket, lagt i et dokument og sendt med emnet "✅ kladde klar". Nu stopper robotten med en tydelig fejlmail.

Øvrige: møder uden dato crashed indsamlingen; `LAST_FA_PROCESSED_MS` kunne skjule møder for altid; email-dedup manglede; `extractTextFromHtml_` smed alle afsnit væk; død kode fjernet; `moveTo()` i stedet for forældet `addFile` .

Fase 5 — filen på `main` var en gammel kopi (`85d8490`)

En ny fil `sf-middelfart-robot-v8.gs` blev oprettet på `main` ud fra en **gammel version fra før alle rettelserne**, hvilket bragte `allAmounts` -fejlen og alle de andre tilbage. Git genkendte v8 som en omdøbning af v7, så en merge løste det og bevarede samtidig det nye modelnavn.

Lære: tag altid udgangspunkt i nyeste version på `main` , aldrig i en lokal kopi.

Fase 6 — "high demand"-nedbruddet (bc7aeae) — den vigtigste

Symptom (eksekveringslog):

```
📄 Fandt 18 sager fra denne uge
🔥 Top-sager: 0   🚫 Mellem-sager: 0   📁 Administrative: 18
❌ Gemini API-fejl: This model is currently experiencing high demand...
Error: Nyhedsbrev-generering fejlede
```

To uafhængige rodårsager:

1. `generateNewsletterWithGemini_` lavede ét `UrlFetchApp.fetch`, kaldte **aldrig** `getResponseCode()`, og havde **intet** `retry` — mens de tre andre Gemini-kald havde `retry`. Det var drift mellem fire næsten ens kopier, ikke design. Kørslen brugte 40 af 360 sekunder; der var rigelig tid til flere forsøg.
2. `analyzeWithGemini_` skrev `{ tldr: "Analyse fejlede", score: 1 }` ved ENHVER fejl. Score 1 = administrativ = **permanent udelukket** fra nyhedsbrevet. En netværksfejl blev til en redaktionel vurdering.

Rettelser: fælles `geminiFetch_` med HTTP-tjek, `retry` og `deadline-vagt`; eksekverings-tidsbudget; fejl skriver ingenting; pr.-række-skrivning; `analyzePendingRows_` + `dailyRepairAnalyses`; dødmandsknap i `generateWeeklyDraft`; kladden gemmes før fakta-tjekket.

Fase 7 — modellen selv (2cfa55c)

Efter fase 6 viste en ny kørsel at 16 sager stod med **tom** score, og at `dailyRepairAnalyses()` ikke kunne reparere dem. Reparationskoden blev gennemgået og er korrekt — så fejlen ligger i selve analyse-kaldet.

To dokumenterede forhold ved `gemini-3.7-flash` forklarer det:

1. Gemini 3.x **tænker altid**. På 3.7-flash kan thinking ikke slås fra (kun `low / medium / high`; `minimal` returnerer en fejl). Tænke-tokens tælles med i `maxOutputTokens`, og `callGeminiJson_` satte **ingen grænse**. Bruger modellen standard-loftet på at tænke, kommer svaret tomt eller afkortet midt i JSON'en tilbage.
2. Schema-bundet JSON på 3.7-flash har en kendt regression hvor den kan gå i løkke på prompts med **mange næsten ens elementer** — præcis mønsteret i en uges kommunale dagsordenspunkter.

Rettelser: `CFG.ANALYSIS_MAX_TOKENS = 8192` på alle tre JSON-kald; `CFG.MODEL_FALLBACKS` så `geminiFetch_` prøver 3.6-flash og dernæst 3.5-flash; uparsbar JSON logger nu svarets første 200 tegn; ny `debugDiagnoseSheet()`.

Thinking-parameteren røres bevidst **ikke**: feltnavnet er forskelligt mellem Interactions API (`thinking_level`) og `generateContent`, og et forkert felt ville give HTTP 400 på hvert eneste kald. Token-plads løser samme problem uden den risiko.

4. Nøglemekanismer i den nuværende kode

Tidsbudget (øverst i filen)

```
const EXEC_START_MS    = Date.now();    // top-level: sættes én gang pr. eksekvering
const EXEC_BUDGET_MS   = 300 * 1000;    // 300 s arbejde → 60 s margin til 360 s-grænsen
const WORST_FETCH_MS   = 70 * 1000;    // konservativt loft for ÉT UrlFetch
```

`timeFor_(ms)` svarer "er der plads til dette?". Bruges før hvert netværkskald og i hver løkke. Forankringen i eksekveringen (ikke i den enkelte funktion) er bevidst: `dailyIngest` kalder `analyzeNewRows_` to gange, og et lokalt ur ville nominelt tillade 9+ minutter.

`geminiFetch_` — ét sted for alle AI-kald

Alle fire kaldsteder (analyse, PDF-analyse, nyhedsbrev, fakta-tjek) går gennem den. Den håndterer: HTTP-kodetjek, retry på `429` / `5xx` / `UNAVAILABLE` / `RESOURCE_EXHAUSTED`, netværks-exceptions, bundet backoff (3 s/6 s), deadline-vagt pr. forsøg, og fallback til reservemodeller. Klassifikationen ser på HTTP-koden og `error.status` — **aldrig** på fejltekstens ordlyd, som Google kan omformulere.

Fakta-tjekket

`collectGroundTruth_` genhenter kildeteksten **friskt fra API'et** (ikke fra arket) for de sager der indgår, plus kommende møder som `kalender`-kilder. `factCheckNewsletter_` beder Gemini klassificere hver påstand som `verified` / `unverified` / `contradicted` ved temperatur 0.0. Resultatet lægges øverst i dokumentet og afspejles i mailens emnelinje:

Tilstand	Emnelinje
Ikke tjekket	⚠ IKKE FAKTA-TJEKKET –
Modsagte påstande	⊘ MODSAGTE PÅSTANDE –
Uverificerede	⚠
Rent	(intet præfiks)

Dødmandsknappen

Er der 0 top-sager, 0 mellem-sager **og** mindst én uanalyseret række, afbrydes kørslen med en dansk forklarende mail. En **ægte** stille uge (alt analyseret, alt score 1-2) rammer ikke denne gren og får stadig sit korte, ærlige nyhedsbrev.

5. Verifikation

Efter enhver kodeændring

```
cp sf-middelfart-robot-v8.gs /tmp/chk.js && node --check /tmp/chk.js
```

Apps Script-filer er ren JavaScript; `node --check` fanger syntaksfejl. Bemærk: Node-API'er findes ikke i Apps Script — brug kun `UrlFetchApp` , `SpreadsheetApp` , `DriveApp` , `DocumentApp` , `PropertiesService` , `Utilities` .

I Apps Script (rækkefølge betyder noget)

Trin	Funktion	Forventet
1	<code>debugTestGemini()</code>	Bekræfter at AI-kaldet overhovedet virker
2	<code>debugDiagnoseSheet()</code>	Viser hvor mange rækker der er analyseret / venter / er forgiftet
3	<code>dailyRepairAnalyses()</code>	Reparerer manglende analyser; kan kræve flere kørsler
4	<code>debugDiagnoseSheet()</code>	Tallet skal være faldet
5	<code>testManualRun()</code>	Kun indsamling (aldrig sammen med nyhedsbrevet)
6	<code>testGenerateNewsletter()</code>	Kladde + fakta-tjek + mail

Efter ændring af triggere skal `setupOnce_createTriggers()` køres igen — den sletter ALLE eksisterende triggere og opretter dem forfra.

Påkrævede Script Properties

`SPREADSHEET_ID` , `INBOX_SHEET_NAME` (default `Inbox`) , `INBOX_LABEL` (Gmail-label), `GEMINI_API_KEY` , `DRAFT_FOLDER_ID` .

6. Uafklarede punkter

- Fase 7-rettelsen er ikke afprøvet.** Reservemodeller og token-plads er pushet, men ikke kørt i Apps Script. Første handling bør være at verificere den: kør `debugTestGemini()` og `dailyRepairAnalyses()` og læs loggen.
- Loggen fra `dailyRepairAnalyses()` er aldrig set.** Diagnosen i fase 7 bygger på dokumenterede modelforhold plus udelukkelse — ikke på en observeret fejlbesked. Loggen indeholder nu Googles egen besked og de første 200 tegn af et ubrugeligt svar, så næste kørsel afgør sagen.
- Reviewet af fase 6-rettelserne nåede kun 17 af 38 kontroller**, før organisationens forbrugsgrænse blev ramt. Tre bekræftede regressioner blev rettet. Ubekræftede fund der kan være værd at se på: manglende tidsvagt i `ingestInboxEmails` ' trådløkke; `analyzePendingRows_` bruger kun kolonne H og genhenter ikke PDF/URL-indhold; efteranalysen tager ældste række først, så en pukkel kan sulte den aktuelle uge.
- GitHub API-adgang er blokeret i dette miljø.** `git push` virker, men PR'er skal oprettes manuelt via `https://github.com/Mweimar2000/maja-sf.dk/compare/main...<branch>` .
- `gemini-3.7-flash` er ny.** Hvis fallback-kæden viser sig at bære byrden i praksis, så overvej at gøre en stabil model til primær i stedet.

7. Metode — sådan blev fejlene fundet

Værd at gentage, fordi den fandt fejl som en enkelt gennemlæsning ikke ville have:

1. **Parallel undersøgelse langs uafhængige akser.** Fire agenter fik hver sin linse på samme fil: API-robusthed, tavs datakorruption, timeout-budget, modeltilgængelighed. De fandt 42 fund, hvor overlappet mellem dem afslørede hvilke der var ægte.
2. **Adversarisk verifikation af hvert fund.** Hvert fund gik til en skeptiker med instruks om at *modbevise* det ved at læse den faktiske kode. Det afviste ca. en femtedel som misforståelser — og fangede kritik af selve rettelsesforslagene, herunder at en naiv fælles hjælper ville have brudt returtypen og gjort hver eneste række til en fejl.
3. **Review af egne ændringer før push.** Samme metode kørt på min egen diff fandt tre regressioner jeg selv havde introduceret — blandt andet at emnelinjen havde mistet advarslen ved modsagte påstande.
4. **Loggen som primærkilde.** Fejlstrengen `✗ Gemini API-fejl:` findes præcis ét sted i filen, og stacktracens linjenumre matchede filen 1:1. Diagnosen var bevist, ikke formodet.

8. Genskabelse fra nul

```
git clone https://github.com/Mweimar2000/maja-sf.dk
cd maja-sf.dk
git log --oneline          # kronologien i afsnit 3
```

1. Opret et Apps Script-projekt og indsæt `sf-middelfart-robot-v8.gs`
2. Udfyld de fem Script Properties (afsnit 5)
3. Opret Google Sheet med kolonneoverskrifter A-O (afsnit 1) og et ark ved navn som `INBOX_SHEET_NAME`
4. Opret en Drive-mappe til kladder og sæt `DRAFT_FOLDER_ID`
5. Kør `setupOnce_createTriggers()` én gang
6. Verificér efter tabellen i afsnit 5

Tonen redigeres i `stilguide.md` og pushes til GitHub — robotten henter den live ved næste kørsel.

`SF_TONE_GUIDE_FALLBACK` i koden er kun en nødudgang hvis GitHub ikke kan nås, og behøver ikke holdes i sync.